SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING

TO : **MR. S YUTE**

FROM : **HOPE KUTENGULE AND MARRIAM HAJI**

REG NO : **BSC/ COM/NE/04/22 AND BSC/COM/NE/28/23**

COURSE CODE : **NET322**

COURSE NAME : **NETWORK PROGRAMMING AND APPLICATION TO DEVELOPMENT**

ASSIGNMENT TITLE : **MOBILE APP DEVELOPMENT ASSIGNMENT 1**

DUE DATE : **8th MAY 2026**

YAML is used for sensor configuration files because it's easy for people to read and write, with a clean structure that avoids too much clutter. Operators can add comments directly in the file, which helps explain why certain settings are chosen. On the sensor-to-server link, Protobuf is preferred because it's a compact binary format that saves bandwidth and speeds up communication and this is important when sensors have limited resources. It also enforces a strict schema, so both sides know exactly what data is being sent and received, reducing errors and supporting compatibility as systems evolve. For public REST APIs, JSON and XML are common because they are text-based, widely supported across programming languages, and easy to debug. JSON is lighter and less verbose, making it quick to work with, while XML is more verbose but offers strong schema validation and extensibility, which is useful for complex contracts. In short, YAML emphasizes human-readability, Protobuf emphasizes performance and schema rigor, and JSON/XML emphasize interoperability and validation for external developers.

REST is a natural fit for a historical-data API because its statelessness and resource-orientation align perfectly with the nature of historical records. Each dataset for example, a year's worth of sensor logs or a specific event archive can be modeled as a resource with unique URLs like sensor id/ readings and clients can retrieve these resources without maintaining session state. This makes REST scalable and simple, since the server doesn't need to remember client context across requests. In contrast, SOAP is more heavyweight, relying on envelopes and strict schemas, which adds verbosity and complexity that isn't necessary for straightforward data retrieval. RPC focuses on invoking remote procedures rather than exposing resources, which tightly couple clients to server-side operations and undermines the loose coupling that REST provides. For historical data, where the primary need is to fetch well defined resources repeatedly and reliably, REST's stateless, resource centric design ensures efficiency, clarity, and broad interoperability. Furthermore, REST integrates naturally with Http features such content negotiation, status codes, cookies and persistent connection. This makes it highly suitable highly suitable for web facing telemetry services accessed by diverse client applications.

This is an I/O bound problem because the main bottleneck is waiting for external operations like network requests, disk reads, or database queries rather than heavy computation. In such cases, using AsyncIO with coroutines and an event loop is preferable to threads: the event loop can efficiently schedule many tasks without the overhead of creating and managing multiple threads, and coroutines allow the program to pause while waiting for I/O and resume later without blocking other tasks. This leads to lower memory usage and better scalability when handling thousands of concurrent connections. Threads, on the other hand, introduce context-switching costs and synchronization complexity, which are unnecessary when tasks spend most of their time idle, waiting for I/O. However, threads or even processes become better fit when the workload is CPU bound for example, performing heavy computations, image processing, or numerical simulations because AsyncIO does not parallelize CPU work. In short, AsyncIO shines for I/O-heavy workloads with high concurrency, while threads or processes are more appropriate when raw computational power or parallel execution is required.

In designing an HTTP based system, content negotiation and persistent connections each have a significant impact. Content negotiation affects the design by requiring the API to support multiple representations of the same resource for example, JSON for lightweight web applications, XML for systems needing schema validation, or CSV for analysts. This means the server must be able to inspect client headers like Accept and deliver the appropriate format, ensuring interoperability across diverse consumers. Persistent connections, on the other hand, influence performance and scalability: by reusing the same TCP connection for multiple requests and responses, the system avoids the overhead of repeatedly opening and closing connections. This is especially beneficial when clients make frequent or bulk requests, such as retrieving large sets of historical data, because it reduces latency and improves throughput. Together, these mechanics shape the design to be both adaptable to client needs and efficient in handling repeated interactions.

References

. *Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine.*

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

*4. Google. Protocol Buffers Documentation. Protocol Buffers Document Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media. 7. Mozilla Developer Network. *HTTP Content Negotiation*.*